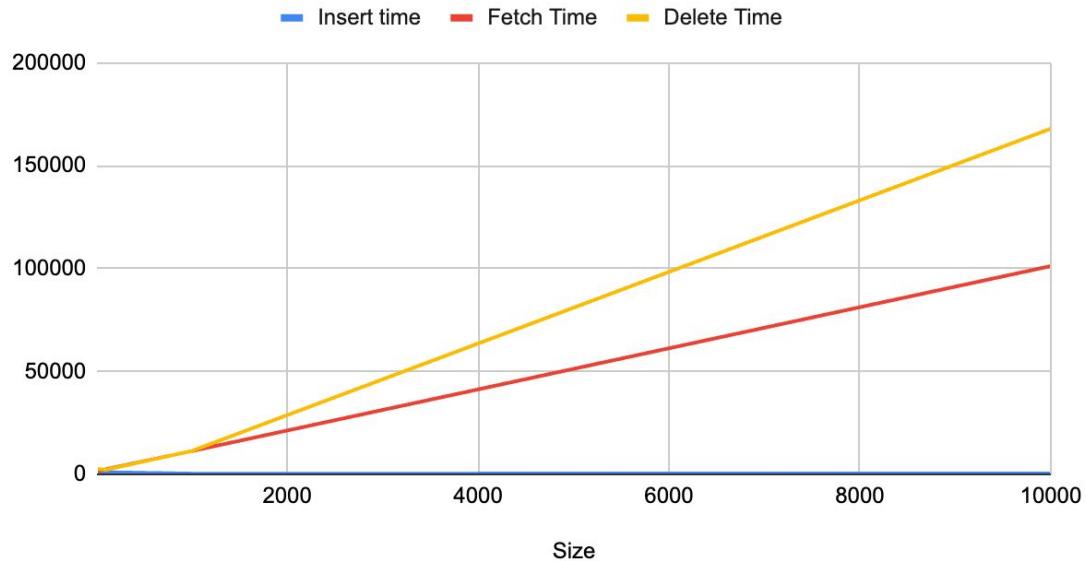


Array Performance

Experimental vs theoretical data

For my code I made 3 different classes and tested them all separately. You won't see all the code for each test because I was reusing the test class to test the objects methods.

Unsorted

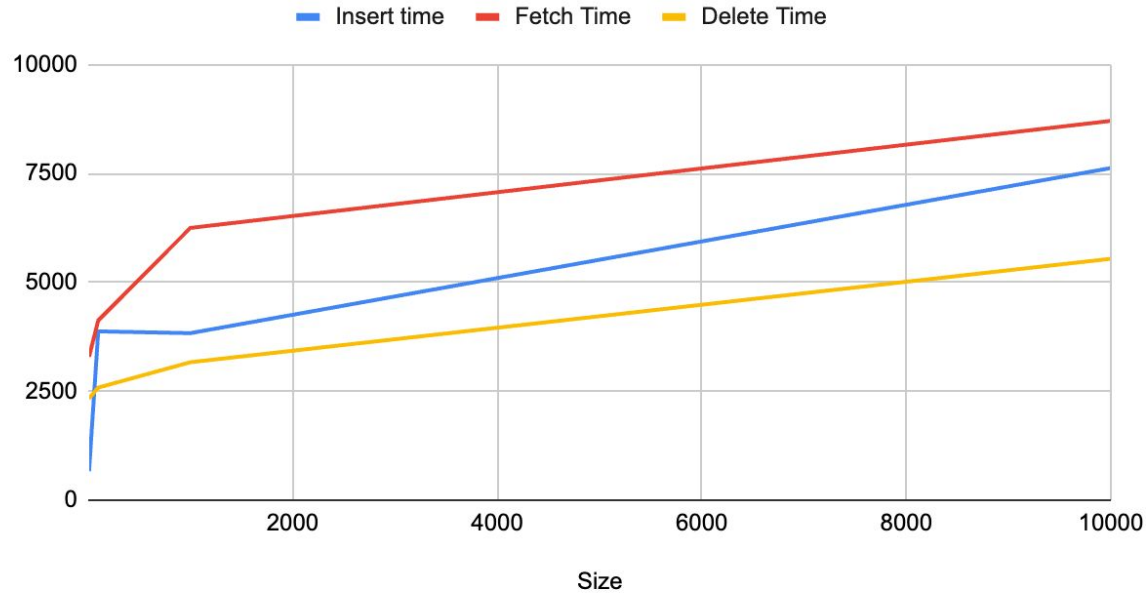


This graph shows that both Delete and Fetch times increase with size in an unsorted array. This is because Big O for these methods is $O(n)$ meaning that the time is dependent on the number of elements in the worst case.

The insertion is constant because its Big O is $O(c)$; the last element is always inserted at the end.

These results line up with the theoretical complexities

Sorted

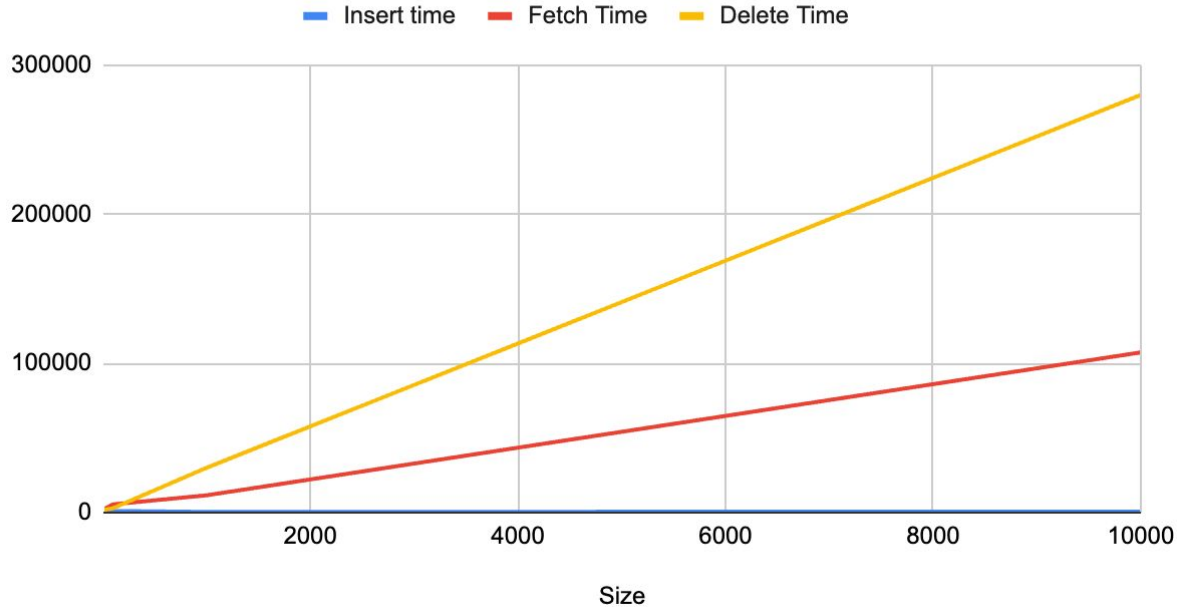


This graph shows that all methods times increase with size. In order to get more accurate data closer to the theoretical complexities I would need to use an even larger data set, which would take lots of time. In the experiment all methods have a big O of $O(n)$ meaning the times are based on size. The fetch algorithm is the only one that should not be $O(n)$; in its theoretical complexity it should be $O(\log n)$ meaning eventually the line levels out and time stops increasing.

My code uses a binary search for the fetch algorithm, the delete method implements this to quickly find the matching key to delete.

The insert method searches and shifts all the elements when it find the correct spot.

Unsorted Optimized



This graph shows that both Delete and Fetch times increase with size in an unsorted array. This is because Big O for these methods is $O(n)$ meaning that the time is dependent on the number of elements in the worst case.

The insertion is constant because its Big O is $O(c)$; the last element is always inserted at the end.

These results line up with the theoretical complexities.

The Unsorted Optimized is the same as just unsorted for Big O, because it is the worst case scenario, however for the best cases the optimized is better.

For my fetch algorithm every time something is fetched it is moved one index over making it closer to the front. Insert is still always at the
The delete method uses the fetch method to find the element to delete so it makes it faster if it was a frequently fetched element.

- Why does the sorted array use binary search for fetch? How does this affect its complexity?

It uses a binary search to because it makes it faster by dividing the rays into halves. This decreases the time to find something as its big o is $O(\log n)$ which is faster than just $O(n)$;

- How does the optimization in Unsorted Optimized Array improve deletion performance?

It allows you to delete an element without shifting every element which would take more time. You simply find the element you are searching for then swap it with the very last element. The decrease the top by 1 essentially deleting the element you searched for.